

Fila de Prioridade (priority_queue)

priority-queue.md

Sumário

Fila de Prioridade (priority_queue)	3
1. A ideia	3
2. As operações	3
3. Para sair o menor primeiro	5
Caminho 1: greater	5
Caminho 2: guardar negativo	5
4. priority_queue de pair e de struct	5
Com pair	6
Com struct e comparador próprio	6
5. A pegadinha: o comparador é “ao contrário”	6
Critérios de desempate	7
6. Onde a fila de prioridade aparece	7
6.1 “A cada passo, pegue o maior (ou menor) disponível”	7
6.2 Os k maiores de uma sequência enorme	7
6.3 Simulações com tempo	8
6.4 Dijkstra (caminho mínimo)	8
7. priority_queue, set ou sort?	8
8. Aplicação: o problema 2370 (Times)	8
O enunciado, em resumo	8
A dica	9
9. Erros comuns	9

Fila de Prioridade (priority_queue)

Este material continua os anteriores de Introdução ao C++, `vector`, Busca Binária, Ordenação e Dois Ponteiros, Pilha e Fila, e Set e Map. No material de pilha e fila vimos duas estruturas em que a ordem de saída é decidida pela **ordem de chegada**: a pilha devolve o mais recente, a fila devolve o mais antigo. Agora vamos conhecer uma fila em que a ordem de chegada **não importa**: sai sempre o **mais importante**.

1. A ideia

Pense na fila de um pronto-socorro. As pessoas chegam em qualquer ordem, mas não são atendidas por ordem de chegada: quem está em estado mais grave passa na frente. Isso é uma **fila de prioridade**: você vai colocando elementos, e toda vez que tira um, sai o de **maior prioridade** entre os que estão lá.

```
push(3)  push(10)  push(1)  push(7)

pop() → 10      pop() → 7      pop() → 3      pop() → 1
```

Repare que a `priority_queue` não guarda tudo ordenado como o `set`. Ela só garante uma coisa: **o elemento do topo é sempre o maior**. Os outros ficam “bagunçados” lá dentro, e é justamente por não se preocupar com a ordem de todos que ela consegue ser tão rápida.

NOTA

Ela vem do cabeçalho `<queue>`, o mesmo da `queue`. Com `#include <bits/stdc++.h>` já está incluída.

2. As operações

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    priority_queue<int> pq;

    pq.push(3);
    pq.push(10);
    pq.push(1);
    pq.push(7);

    cout << pq.top() << '\n';    // 10 (olha o maior, NÃO remove)
    pq.pop();                    // remove o 10
    cout << pq.top() << '\n';    // 7

    cout << pq.size() << '\n';    // 3

    while (!pq.empty()) {
        cout << pq.top() << ' ';    // 7 3 1
        pq.pop();
    }
}
```

cpp

Função	O que faz
<code>pq.push(x)</code>	Insere <code>x</code>
<code>pq.pop()</code>	Remove o elemento de maior prioridade (não devolve ele)
<code>pq.top()</code>	Devolve o de maior prioridade, sem remover
<code>pq.size()</code>	Quantos elementos há
<code>pq.empty()</code>	<code>true</code> se está vazia

Se você prestou atenção no material de pilha, percebeu que os nomes são **iguais aos da** `stack`: `push`, `pop`, `top`. A diferença é o que o `top()` devolve: na pilha, o **último que entrou**; aqui, o **maior**.

E os mesmos avisos valem:

ATENÇÃO

`pop()` **não devolve nada**. Leia com `top()` antes, depois remova.

CUIDADO

`top()` ou `pop()` em uma fila de prioridade **vazia** faz o programa se comportar de forma imprevisível. Confira `empty()` antes.

Também não existe `clear()`, nem `pq[i]`, nem `for (auto x : pq)`. Para “ver tudo”, só tirando do topo, o que esvazia a fila:

```
while (!pq.empty()) {
    int x = pq.top();
    pq.pop();
    // ... usa x ...
}
```

cpp

Esse laço devolve os elementos **do maior para o menor**. Ou seja, colocar tudo em uma `priority_queue` e tirar um a um é mais uma forma de ordenar.

3. Para sair o menor primeiro

O padrão é “maior primeiro”. Para inverter, há dois caminhos.

Caminho 1: `greater`

```
priority_queue<int, vector<int>, greater<int>> pq; // sai o MENOR primeiro
```

cpp

A declaração é feia, mas é decoreba: `<tipo, vector<tipo>, greater<tipo>>`. O `greater<int>` é o mesmo que você viu no `sort` para ordem decrescente. Parece contraditório usar “greater” para sair o menor, e é mesmo: é uma pegadinha do C++ que vamos explicar na seção 5. Por enquanto, decore.

Caminho 2: guardar negativo

Como o maior dos negativos é o menor dos positivos, dá para usar a `priority_queue` normal e trocar o sinal na entrada e na saída:

```
priority_queue<int> pq;

pq.push(-x);           // ao inserir, nega
int menor = -pq.top(); // ao ler, nega de volta
pq.pop();
```

cpp

É um truque rápido de digitar, mas fácil de esquecer um dos sinais. Prefira o caminho 1 quando tiver tempo de escrever a declaração.

4. `priority_queue` de `pair` e de `struct`

Com pair

Como no `sort` e no `set`, a `priority_queue` usa o `<` do tipo. Um `pair` compara pelo `first` e desempata pelo `second`. Então:

```
priority_queue<pair<int, string>> pq;

pq.push({3, "ana"});
pq.push({9, "bia"});
pq.push({3, "caio"});

// sai: {9, "bia"}, {3, "caio"}, {3, "ana"}
```

cpp

Sai primeiro o maior `first`. Entre `first` iguais, o **maior** `second` ("caio" é alfabeticamente maior que "ana"), porque a fila sempre entrega “o maior” segundo o `<` do `pair`.

Um uso muito comum: guardar `{valor, índice}` para saber **quem** era o maior, e não só o valor.

Para “menor `first` primeiro” com `pair`, o `greater` funciona igual:

```
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
```

cpp

Com struct e comparador próprio

Quando a regra de prioridade é diferente de “compare pelo `<` do tipo”, você escreve um **comparador**, como no material de ordenação. Aqui ele precisa ser uma `struct` com `operator()`:

```
struct Aluno {
    string nome;
    int nota;
};

struct Comparador {
    bool operator() (const Aluno& a, const Aluno& b) {
        return a.nota < b.nota; // sai primeiro quem tem MAIOR nota
    }
};

priority_queue<Aluno, vector<Aluno>, Comparador> pq;
```

cpp

A declaração segue a mesma decoreba: `<tipo, vector<tipo>, Comparador>`.

5. A pegadinha: o comparador é “ao contrário”

Esta é a parte que mais confunde, então vale ler devagar.

No `sort`, o comparador responde: “a deve vir antes de b?”. Quem o comparador coloca antes, aparece primeiro no vetor.

Na `priority_queue`, o comparador responde: “a tem prioridade menor que b?”. Quem o comparador considera “menor” sai por **último**. Em outras palavras, a fila entrega primeiro quem o comparador coloca **por último**.

Por isso:

- `return a.nota < b.nota;` faz sair primeiro a **maior** nota. O comparador diz “nota menor tem prioridade menor”, então a maior fica no topo.
- `return a.nota > b.nota;` faz sair primeiro a **menor** nota.
- `greater<int>` diz “a é maior que b” = “a tem prioridade menor quando é maior”, logo o menor sai primeiro. Era esse o mistério da seção 3.

Uma forma de nunca errar: escreva o comparador como se fosse para o `sort`, na ordem em que você **quer que as coisas saiam**, e depois **inverte** o sinal da comparação. Ou, mais simples: escreva de um jeito, teste com três valores e, se sair invertido, troque `<` por `>`.

Critérios de desempate

Funciona como no `sort`, com um `if` por critério, só que com a lógica invertida em todos eles. Por exemplo, “sai primeiro quem tem maior nota; entre notas iguais, quem tem o nome alfabeticamente menor”:

```
struct Comparador {  
    bool operator() (const Aluno& a, const Aluno& b) {  
        if (a.nota != b.nota) return a.nota < b.nota; // maior nota primeiro  
        return a.nome > b.nome; // menor nome primeiro  
    }  
};
```

cpp

Leia cada linha como “quando a tem prioridade **menor** que b”: nota menor, ou, com notas iguais, nome maior.

6. Onde a fila de prioridade aparece

6.1 “A cada passo, pegue o maior (ou menor) disponível”

É a frase que dispara a estrutura. Problemas em que você repete muitas vezes “pega o mais urgente, faz alguma coisa, talvez coloca algo novo”. Exemplos: juntar as duas menores pedras várias vezes; atender sempre o pedido mais importante; escolher sempre o aluno mais habilidoso que sobrou.

6.2 Os `k` maiores de uma sequência enorme

Se você lê milhões de números e só quer os `k` maiores, uma fila de prioridade “menor primeiro” com no máximo `k` elementos resolve: insere cada número e, se passou de `k`, remove o menor. O que sobrar são os `k` maiores.

```
priority_queue<int, vector<int>, greater<int>> pq;
for (int x : numeros) {
    pq.push(x);
    if ((int) pq.size() > k) pq.pop();    // joga fora o menor
}
```

cpp

6.3 Simulações com tempo

“Vários eventos vão acontecer, cada um em um instante; processe-os em ordem de tempo, e processar um evento pode criar outros.” Fila de prioridade de `{tempo, evento}` com o menor tempo primeiro.

6.4 Dijkstra (caminho mínimo)

O uso mais famoso: achar o caminho mais curto em um mapa com estradas de tamanhos diferentes. A fila guarda `{distância, cidade}` e sempre entrega a cidade mais próxima ainda não visitada. Vamos ver em um material de grafos; por enquanto, basta saber que **Dijkstra = fila de prioridade com menor primeiro**.

7. priority_queue, set ou sort?

As três conseguem “me dê o maior”. Quando usar cada uma:

Situação	Use
Todos os dados chegam de uma vez e você só quer a ordem	<code>sort</code>
Inserções e retiradas do maior/menor se misturam ao longo do programa	<code>priority_queue</code>
Além do maior/menor, você precisa remover um elemento qualquer , ou saber se um valor existe, ou pegar “o próximo maior que x”	<code>set</code> (ou <code>multiset</code>)

O `set` faz tudo o que a `priority_queue` faz (o menor é `*s.begin()`, o maior é `*s.rbegin()`) e ainda mais. Mas a `priority_queue` é mais simples de usar, mais rápida, e aceita repetidos sem precisar de `multiset`. Se você só precisa de `push`, `top` e `pop`, ela é a escolha certa.

8. Aplicação: o problema 2370 (Times)

O enunciado, em resumo

São dados N alunos, cada um com nome e um nível de habilidade (todos diferentes), e um número T de times. Os times escolhem alternadamente: o time 1 pega o aluno mais habilidoso disponível, depois o time 2 pega o mais habilidoso entre os que sobraram, ..., o time T , e volta ao time 1, até acabarem os alunos. Imprima cada time, com os nomes dos seus jogadores em **ordem alfabética**, e uma linha em branco depois de cada time.

A dica

“Pegar o mais habilidoso entre os que sobraram”, repetidamente, até acabar: é a frase da seção 6.1. Pense em qual informação cada elemento da fila precisa carregar para que, ao sair, você saiba **quem** é o aluno, e não só a habilidade dele. O `pair` da seção 4 ajuda.

Depois de decidir quem vai para cada time, você precisa imprimir cada time em ordem alfabética. Isso é um problema separado: você já tem ferramentas que guardam nomes em ordem (pense no material de set e map) ou que ordenam uma lista pronta. Escolha uma.

Só cuidado com um detalhe de saída: a linha em branco vem **depois de cada time**, inclusive do último.

9. Erros comuns

- Achar que `pop()` devolve o elemento. Ele só remove. Leia com `top()` antes.
- Chamar `top()` ou `pop()` com a fila **vazia**. Confira `empty()`.
- Procurar `clear()`, `pq[i]` ou `for (auto x : pq)`. Não existem. Se você precisa disso, quer um `vector` ou um `set`.
- Esquecer que o padrão é **maior primeiro** e imprimir tudo ao contrário.
- Escrever `priority_queue<int, greater<int>>` (faltou o `vector<int>` no meio). A ordem é sempre `<tipo, vector<tipo>, comparador>`.
- Escrever o comparador com a lógica do `sort` e obter a ordem invertida. Na `priority_queue`, o comparador diz quem tem prioridade **menor**. Teste com três valores.
- No truque do negativo, esquecer de negar de volta ao ler, ou negar um valor que não cabe em `int` (guarde em `long long` se os números forem grandes).
- Usar `priority_queue` para remover um elemento que **não** é o do topo. Ela não faz isso; use `set/multiset`.
- Declarar a fila fora do laço de casos de teste e esquecer de esvaziá-la entre um caso e outro.